

Laboratorio #1

Análisis

- 1. ¿Qué clases son necesarias para representar el problema planteado?**
- 2. ¿Qué propiedades y métodos tendrá cada clase?**
- 3. ¿Qué tipo deben tener las propiedades y métodos de cada clase?**
- 4. ¿Qué información deberá almacenarse dentro de la colección dinámica?**
- 5. ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList?**
- 6. ¿Cuáles deben ser los modificadores de visibilidad de los miembros de cada clase?**
- 7. ¿Qué parámetros serán requeridos por los métodos de sus clases?**
- 8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?**
- 9. ¿Cómo identificará una orden específica dentro de la colección?**

A través del número de la orden, su principal identificador.

- 10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?**
- 11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?**

Se podría recorrer por cada orden comparando la placa registrada en cada orden.

- 12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?**

Se podría recorrer las ordenes con un for-each acumulando el atributo de costo.

- 13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?**

Recorriendo cada una de las ordenes, guardando el primer costo y luego realizar una comparación por cada costo para identificar cual es superior, si alguno es superior se guardará y será el nuevo que se compare con los demás costos.

- 14. ¿Qué situaciones dentro del programa pueden producir una excepción?**

Al registrar una orden con un número ya existente, una orden con nombre del propietario, placa o descripción vacíos, una orden con costo estimado menor o igual a 0.

- 15. ¿Qué operaciones serán protegidas utilizando try-catch?**

Todas aquellas en las que se necesite que el usuario ingrese algún dato, las clases “leer...”, se utilizaría el try-catch para evitar que los datos que se ingresen no sean los adecuados.

16. ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?

Con la operación finally se puede mostrar un mensaje final en el de a entender que las operaciones realizadas por resultaron bien, algo como “¡Proceso exitoso!”.

Tabla de propiedades:

#	Tipo de dato	Nombre	Visibilidad	Propósito	Clase
1	int	numOrden	Private	El número asignado a la orden	Orden
2	String	nombre	Private	Guarda el nombre del propietario del carro	Orden
3	String	placa	Private	Guarda la placa del vehículo asociado a la orden	Orden
4	String	descripcion	Private	Guarda la descripción del servicio solicitado	Orden
5	double	costo	Private	Guarda el costo estimado del servicio	Orden
6	List <OrdenSer>	ordenes	Private	Para guardar las órdenes de servicio	SistemaOrden
7	Scanner	scanner	Private	Un scanner para leer lo que el usuario ingrese.	VistaOrden

Tabla de métodos:

#	Tipo de retorno	Nombre	Visibilidad	Propósito	Clase	Atributos
2	int	getNumOrden	Public	getter del número de la orden	Orden	Ninguno
3	String	getNombre	Public	getter del nombre del propietario	Orden	Ninguno
4	String	getPlaca	Public	getter de la placa del vehículo	Orden	Ninguno
5	String	getDescripcion	Public	Retorna la descripción del servicio	Orden	Ninguno
6	double	getCosto	Public	Retorna el costo estimado	Orden	Ninguno
7	void	setDescripcion	Public	Modifica la descripción del servicio de una orden ya registrada	Orden	descripcion
8	void	setCosto	Public	Modifica el costo estimado de una orden ya registrada	Orden	costo
9	—	GestorOrdenes (constructor)	Public	Pone la colección de órdenes como un ArrayList vacío	SistemaOrden	Ninguno
10	boolean	registrarOrden	Public	Valida los datos ingresados y agrega una nueva orden a la colección	SistemaOrden	numeOrden nombre placa descripcion costo
11	OrdenServicio	buscarOrden	Public	Busca una orden por número y la retorna	SistemaOrden	numeroOrden
12	void	modificarOrden	Public	Modifica la descripción o costo de una orden	SistemaOrden	numOrden descServicio costoEstimado

13	void	cancelarOrden	Public	Elimina de la colección la orden con el número indicado	SistemaOrden	numeroOrden
14	List <OrdenSer>	consultaOrdenesPlaca	Public	Retorna todas las órdenes asociadas a una placa específica	SistemaOrden	placa
15	double	calculoCostoTotal	Public	Suma los costos estimados de todas las órdenes	SistemaOrden	Ninguno
16	double	calculoCostoPromedio	Public	Calcula el promedio de costo de las órdenes	SistemaOrden	Ninguno
17	OrdenServicio	consultaMayorCosto	Public	Recorre la lista y devuelve la orden con el costo estimado más alto	SistemaOrden	Ninguno
18	int	consultaCantidadOrden	Public	Retorna la cantidad de órdenes registradas actualmente	SistemaOrden	Ninguno
19	List <OrdenSer>	consultaOrden	Public	Retorna la lista completa de órdenes para consultas o impresión	SistemaOrden	Ninguno
20	boolean	existeOrden	Private	Método que detecta si una orden ya existe	SistemaOrden	numOrden
21	void	main	Public	Inicio del programa	Main	args: String[]
22	void	mostrarMenu	Private	Despliega en consola las 10 opciones disponibles	VistaOrden	Ninguno
23	int	leerMenu	Private	Lee la opción del menú ingresada por el usuario	VistaOrden	Ninguno

24	void	mostarTodasOrdenes	Private	Solicita a gestor todas las órdenes y las imprime	VistaOrden	Ninguno
25	void	mostrarOrden	Private	Solicita el número de orden y la imprime	VistaOrden	Ninguno
26	void	leerModificacion	Private	Lee la modificación que se quiera realizar	VistaOrden	Ninguno
27	void	leerCancelarOdern	Private	Lee la orden que se quiera eliminar	VistaOrden	Ninguno
28	void	leerConsultarPlaca	Private	Lee la placa que se quiera consultar	VistaOrden	Ninguno
29	void	mostrarCostos	Private	Muestra el valor total y el costo promedio de las órdenes	VistaOrden	Ninguno
30	void	mostrarMayorCosto	Private	Imprime la orden con el costo estimado más alto	VistaOrden	Ninguno
31	void	mostrarCantidadOrdenes	Private	Muestra cuántas órdenes están registradas actualmente	VistaOrden	Ninguno